

Implementação e Avaliação de Substituição de Cache Baseada em Perceptron

Fernando Infantini Satte Alam Nogueira, Ronaldy da Silva Gelos

2026/1

Resumo

Este trabalho apresenta o projeto, implementação e avaliação de um algoritmo de substituição de blocos em cache baseado em Inteligência Artificial (AIRA), especificamente utilizando o modelo de aprendizado de máquina *Perceptron*. O sistema de memória hierárquica (L1 e L2) foi projetado em Verilog. O módulo de cache foi validado funcionalmente por simulação e sintetizado logicamente tendo como alvo a arquitetura de um FPGA Cyclone III. O objetivo principal é superar o desempenho do algoritmo padrão LRU em cenários de pressão de memória. Os resultados de taxa de acerto (*Hit Rate*), área lógica e temporização são comparados sob diferentes configurações (LRU e *Perceptron*), demonstrando a viabilidade e os custos estruturais do preditor neural em testes de estresse de memória e variações de carga de trabalho. Para fins de reprodutibilidade, todos os códigos-fonte RTL, ambientes de simulação e scripts de análise encontram-se disponíveis publicamente no repositório Git do projeto.

1 Introdução

O abismo de desempenho entre o processador e a memória principal motiva a busca por políticas eficientes de gerenciamento de cache. Tradicionalmente, algoritmos como o LRU (*Least Recently Used*) são utilizados, mas falham em capturar padrões complexos de reuso de memória, especialmente em cargas de trabalho com grandes saltos de endereçamento ou padrões de *streaming*.

Este trabalho propõe a substituição do LRU por um algoritmo inteligente baseado em Inteligência Artificial (AIRA) [2]. O foco deste projeto é o uso do algoritmo de aprendizado *Perceptron* para predição de reuso de blocos [1], buscando maximizar a eficiência da hierarquia de memória, limitando-se às restrições físicas e lógicas estimadas de um FPGA Cyclone III. O sistema foi projetado de forma modular visando facilitar uma futura prototipação em hardware físico. Todos os arquivos de configuração, dados paramétricos e logs de execução que embasam este relatório podem ser acessados via repositório Git.

2 Trabalhos Relacionados

A utilização de redes neurais em hardware preditivo ganhou tração com o trabalho seminal de Jiménez e Lin [1], que propuseram a predição de desvios (*branch prediction*)

dinâmica utilizando *Perceptrons*. Eles demonstraram que preditores neurais podem superar preditores baseados em tabelas de histórico global bimodal devido à capacidade de ponderar múltiplas variáveis de entrada simultaneamente.

Mais recentemente, Teran et al. [2] expandiram esse conceito da predição de fluxo de controle para a predição de reuso de dados em cache. O algoritmo proposto por eles mapeia características (*features*) da memória, como o *Program Counter* (PC) e bits de endereço, para treinar pesos dinamicamente. Nosso trabalho utiliza essas fundações teóricas para construir uma implementação viável e avaliável em nível de Registro de Transferência (RTL).

3 Fundamentação Teórica

3.1 O Modelo Perceptron Clássico

O *perceptron* é um dos modelos mais simples de redes neurais, capaz de aprender funções booleanas separáveis linearmente. Ele calcula o produto escalar entre um vetor de pesos e um vetor de entradas. A saída y é determinada pela soma ponderada das entradas, acrescida de um viés (*bias*), formulada como:

$$y = w_0 + \sum_{i=1}^n x_i w_i$$

Em implementações teóricas, o treinamento ocorre incrementando ou decrementando os pesos com base no acerto ou erro da predição em relação a um limiar de treinamento θ [1]. Contudo, visando a otimização de recursos lógicos neste projeto, nossa implementação simplifica esse processo ao dispensar o limiar, atualizando os pesos sinápticos sempre em incrementos ou decrementos unitários (+1 ou -1) a cada acesso.

3.2 Perceptron Aplicado à Cache (AIRA)

Neste projeto, o *perceptron* prevê o reuso de blocos na cache [2]. Os recursos de entrada (*features*) adotados para formar a predição foram estritamente selecionados como: 2 bits oriundos da *Tag* de endereço, uma flag indicadora de reuso (*Reused*) e o *Bias*. A predição final é dada pelo somatório dos pesos, resultando em um valor de *score*. Se o *score* for menor que zero (negativo), o bloco é predito como "sem reuso" (morto). Durante uma falha de cache (*cache miss*), o algoritmo compara os *scores* de todas as vias do conjunto e seleciona como vítima para substituição o bloco que apresentar o menor *score*, indicando a maior probabilidade de não ser mais utilizado.

4 Detalhamento da Arquitetura de Hardware (RTL)

Para viabilizar o mapeamento lógico do algoritmo de Inteligência Artificial dentro de premissas realistas de clock, a arquitetura foi descrita inteiramente em Verilog com foco em paralelismo espacial e modularidade.

4.1 Parametrização e Modularidade

Embora o foco da análise seja a Inteligência Artificial (IA), o projeto foi desenvolvido garantindo alta parametrizabilidade, permitindo instanciar hierarquias de cache variadas

sem alteração estrutural do código. A Tabela 1 lista os parâmetros base, facilitando o reúso do IP (*Intellectual Property*) gerado.

Tabela 1: Principais Parâmetros de Configuração RTL

Parâmetro Verilog	Descrição Física	Escopo de Teste Típico
CACHE_SIZE	Capacidade total da cache	4 KB (L1) / 32 KB a 64 KB (L2)
BLOCK_SIZE	Tamanho da linha de cache	32 Bytes (L1) / 64 Bytes (L2)
NUM_WAYS	Grau de associatividade	2, 4 (L1) / 8, 16 (L2)
ADDR_WIDTH	Largura do barramento	32 bits (Padrão do Sistema)

4.2 Visão Geral do Datapath e Top-Level

A arquitetura `top_cache_total.v` engloba dois níveis de memória: Caches L1 separadas para Dados e Instruções, e uma Cache L2 Unificada. A Figura 1 ilustra a topologia geral da modelagem proposta em nível RTL.

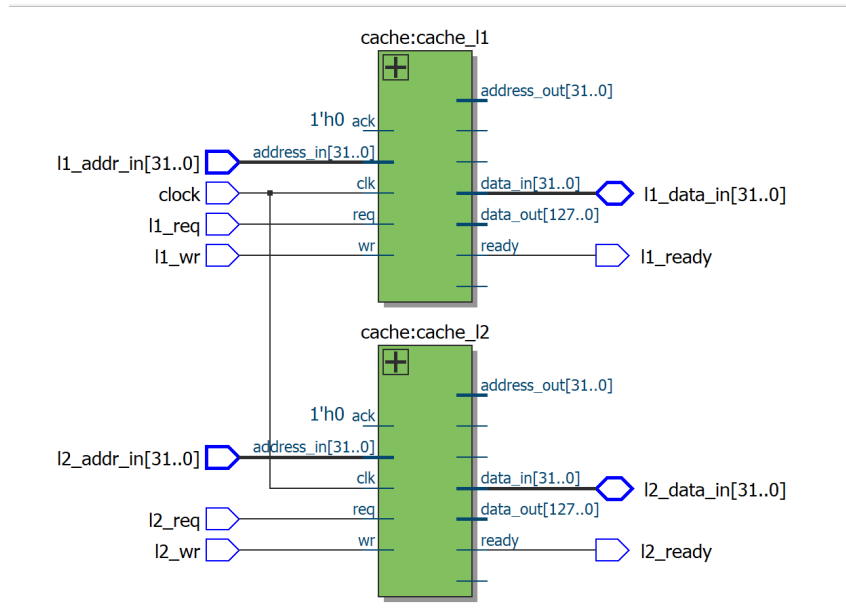


Figura 1: Diagrama de blocos da hierarquia de cache L1 e L2.

A validação dessa estrutura foi confirmada durante a compilação no Quartus II. O RTL Viewer gerou a esquematização de blocos traduzida a partir do código fonte, garantindo a instanciação correta dos módulos de decisão (*Perceptron* vs LRU), conforme demonstrado na Figura 2.

4.3 Otimizações de Hardware no Perceptron

Uma implementação ingênua do *Perceptron* exigiria operações de ponto flutuante e acumulação sequencial via multiplicadores, inviabilizando seu uso em hardware de alta frequência. Para contornar isso, otimizações severas foram projetadas no módulo `perceptron.v`:

Flow Summary	
Flow Status	Successful - Sat Jul 11 20:46:51 2026
Quartus II 64-Bit Version	13.1.0 Build 162 10/23/2013 SJ Web Edition
Revision Name	perceptron_synth
Top-level Entity Name	top_cache_total
Family	Cyclone III
Device	EP3C25F324C6
Timing Models	Final
Total logic elements	9,990 / 24,624 (41 %)
Total combinational functions	9,132 / 24,624 (37 %)
Dedicated logic registers	4,608 / 24,624 (19 %)
Total registers	4608
Total pins	135 / 216 (63 %)
Total virtual pins	0
Total memory bits	32,896 / 608,256 (5 %)
Embedded Multiplier 9-bit elements	0 / 132 (0 %)
Total PLLs	0 / 4 (0 %)

Figura 2: Visão RTL extraída da ferramenta de síntese Quartus II.

- **Seleção de Features Otimizada:** O hardware avalia exclusivamente 4 sinais: `p_Bias`, `p_tag_0`, `p_tag_1` e `p_Reused`, limitando a quantidade de pesos a serem processados por ciclo.
- **Pesos Inteiros Saturados:** Os pesos sinápticos são armazenados em registradores como inteiros com sinal de largura fixa (ex: 6 bits), evitando overflow via saturação e sendo atualizados de 1 em 1.
- **Multiplicação via Portas XNOR:** Como as entradas (*features*) do *perceptron* são bipolares (+1 ou -1), elas são representadas logicamente por apenas 1 bit (0 ou 1). Isso permite substituir os custosos blocos multiplicadores de hardware por uma lógica simples envolvendo portas XNOR, que determinam o sinal (adição ou subtração) do peso atualizado.
- **Árvore de Somadores:** O cálculo do score final $\sum x_i w_i$ foi paralelizado através de uma estrutura em árvore de somadores combinacionais, reduzindo a latência logaritmicamente. A topologia lógica projetada pode ser vista na Figura 3.
- **Árvore de Comparadores:** Para escolher a vítima (bloco com maior indício de não-reúso), os *scores* de todas as vias são avaliados simultaneamente.

4.4 Estrutura de Treino e Inferência

O núcleo preditivo funciona alimentando o estado atual da memória nas camadas de inferência. A Figura 4 exibe a esquematização detalhada do fluxo de dados interno do

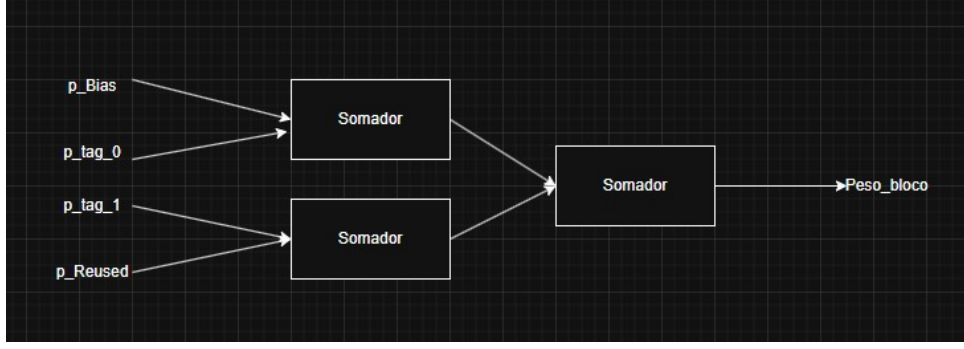


Figura 3: Estrutura lógica da Árvore de Somadores implementada para inferência rápida.

perceptron implementado.

A partir do diagrama, nota-se a divisão fundamental entre o caminho de dados de *feedback* (treinamento) e o caminho preditivo (inferência). Os blocos superiores representam a lógica de treino, que recebe os sinais de exclusão (*excluido_bias*, *excluido_tag0*, etc.) e a flag de acerto (*target_hit*) para atualizar os pesos internos. Em paralelo, a parte inferior realiza a inferência alimentando as instâncias de vias (*via_0*, *via_1*), que injetam seus resultados no módulo comparador (*compara_scores*) para emitir a decisão final de qual bloco deve ser substituído.

Para garantir que o processo de atualização de pesos minimize o atraso combinacional, a lógica de treinamento foi fortemente simplificada. A Figura 5 detalha esse nível lógico.

Nesta etapa, a operação matemática de multiplicação de sinais foi abstraída para uma porta XNOR. Os sinais binários alvo (*target*) e a característica do bloco avaliado (*dado_bloco_miss*) são comparados logicamente. O resultado dessa comparação alimenta um somador simples, determinando se o peso anterior (*peso_velho*) será incrementado ou decrementado. Isso estabelece imediatamente o novo peso sináptico atualizado (*peso_novo*) para o próximo ciclo de clock.

4.5 Cargas de Trabalho e Metodologia de Validação

A validação do sistema operou em duas frentes complementares, utilizando uma abordagem orientada a rastros (*trace-driven simulation*). Para garantir uma avaliação robusta sob diferentes perfis de acesso, cinco *benchmarks* escritos em linguagem C foram executados. Suas características intrínsecas de localidade espacial e temporal estão descritas na Tabela 2.

Tabela 2: Caracterização dos Benchmarks Utilizados

Benchmark	Padrão de Acesso	Localidade Predominante
<i>Streaming</i>	Sequencial contínuo	Alta Espacial, Nula Temporal
<i>Matrix Transpose</i>	Saltos regulares e longos	Baixa Espacial, Baixa Temporal
<i>Matrix Conv.</i>	Varredura 2D sobreposta	Alta Espacial, Alta Temporal
<i>Linked List</i>	Perseguição de ponteiros	Pseudo-aleatória (Baixa em ambas)
<i>Pattern Search</i>	Leitura cíclica/repetida	Moderada Espacial, Alta Temporal

A execução completa desses *benchmarks* atuou como modelo de referência (*Golden Model*). A partir dela, foram extraídas nativamente todas as estatísticas globais de *hit/miss*

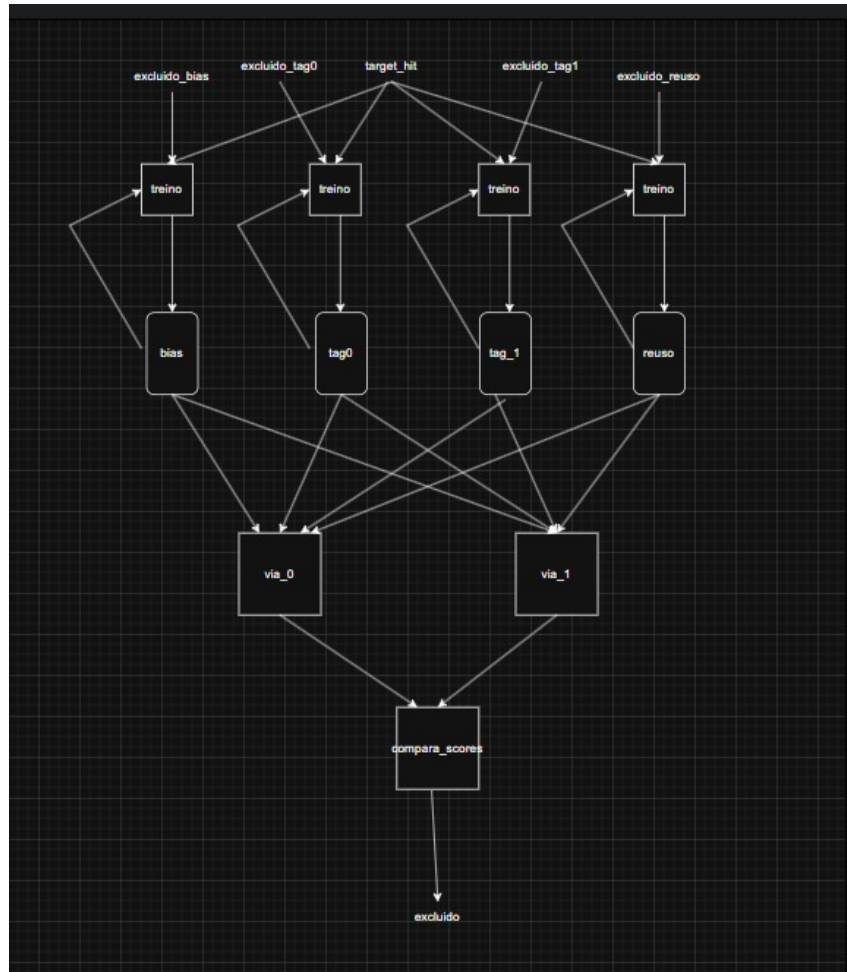


Figura 4: Diagrama estrutural do fluxo de treinamento e inferência no algoritmo AIRA.

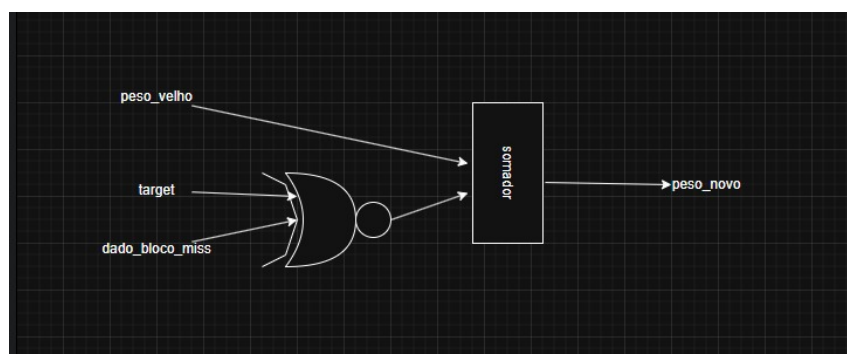


Figura 5: Lógica combinacional da função de treino: atualização de pesos via porta XNOR e somador.

apresentadas neste trabalho. Simultaneamente, os acessos à memória gerados por essas execuções foram capturados e convertidos em vetores de estímulos.

O foco manteve-se estritamente na análise temporal através das formas de onda (*waveforms*) extraídas do simulador. Essa estratégia garantiu a visualização exata das transições de estados, confirmando ciclo a ciclo a estabilidade dos sinais de controle, as decisões de substituição e o cálculo matemático dos pesos, assegurando que a implementação física estivesse perfeitamente alinhada com o modelo teórico.

5 Resultados e Discussões

5.1 Validação Funcional

A Figura 6 comprova a capacidade do hardware de realizar a inferência no momento de um acesso à memória. Através da análise rigorosa da forma de onda (*waveform*), observa-se a latência determinística do *datapath* para calcular os novos pesos após as etapas de *hit/miss*.

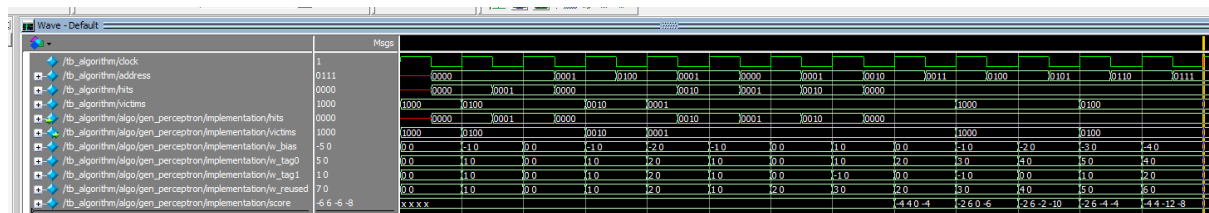


Figura 6: Formas de onda simuladas demonstrando inferência e atualização dos pesos.

5.2 Impacto Físico em Hardware (Custo x Benefício)

A Tabela 3 resume os dados extraídos dos relatórios de compilação do Quartus II. A substituição do LRU por Inteligência Artificial exige recursos substanciais de hardware para manter a alta taxa de processamento de informações contextuais da memória.

Tabela 3: Estimativa de Custo Físico e Desempenho no Benchmark *Matrix Transpose*

Configuração (L1 - L2)	Hit Rate Global	Impacto Hit	Área (LEs)	Impacto Área	Fmax (MHz)	Impacto Fmax
LRU - LRU (Base)	46,87%	-	4.385 un.	-	90,92 MHz	-
Perceptron - LRU	47,27%	+0,40%	6.684 un.	+52,4%	60,02 MHz	-34,0%
LRU - Perceptron	53,37%	+6,50%	8.030 un.	+83,1%	30,64 MHz	-66,3%
Perceptron - Perceptron	54,19%	+7,32%	10.848 un.	+147,3%	28,93 MHz	-68,1%

Aplicar o *Perceptron* apenas na Cache L2 mostrou ser o *sweet spot* da arquitetura, entregando a maior parte do ganho de precisão preditiva (+6,50%) sem o custo dramático de área e sem a acentuada perda de frequência de clock que ocorreria ao instanciar a IA em ambos os níveis simultaneamente.

5.3 Análise de Sensibilidade a Diferentes Cargas de Trabalho

O comportamento de algoritmos puramente históricos é limitado. O LRU padrão é praticamente cego a padrões de acesso que superam a capacidade do conjunto (*set*) da cache, o que degrada severamente o desempenho em aplicações com saltos estruturados. O *Perceptron* resolve parte deste problema, pois consegue correlacionar as assinaturas matemáticas das *Tags* de endereço para mapear esses padrões de "varredura e descarte".

A Figura 7 compila a análise de sensibilidade detalhando as taxas de acerto perante variações paramétricas de hardware para os cinco *benchmarks* testados. Os impactos quantitativos exatos dessas políticas na hierarquia de memória estão consolidados na Tabela 4.

Conforme os dados revelam, o contraste de desempenho está diretamente atrelado à previsibilidade do padrão de acesso. O caso de maior sucesso da abordagem neural ocorre no benchmark *Matrix Transpose*, onde a taxa de acerto global saltou de 46,87% (LRU-LRU) para 54,19% (Perceptron-Perceptron). Nesse cenário, os saltos de endereçamento são regulares, mas longos o suficiente para estourar a capacidade temporal do LRU; o *Perceptron*, contudo, obteve sucesso ao identificar a assinatura dos blocos de descarte na Cache L2, aumentando a eficiência das substituições.

Em contrapartida, cenários com altíssima localidade temporal e espacial, como o *Matrix Convolution*, evidenciam as limitações do preditor estático. Nessa carga de trabalho, o algoritmo LRU tradicional já atinge uma eficiência quase ideal na Cache L1 (97,44%). Ao forçar a substituição via *Perceptron* no primeiro nível, a taxa de acerto decaiu para 88,66%, reduzindo o rendimento global para 98,29%. Esse fenômeno ocorre porque a constante atualização dos pesos sinápticos introduz uma "volatilidade" prejudicial em loops altamente repetitivos, onde o histórico simples do LRU é superior.

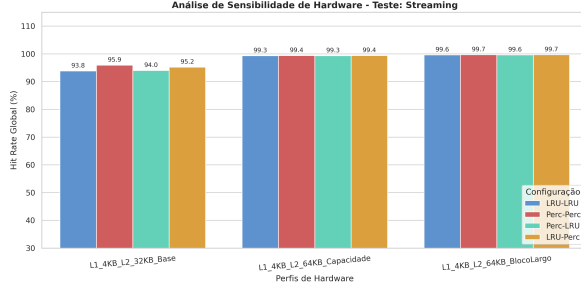
Por fim, em algoritmos com encadeamento pseudo-aleatório, como o *Linked List*, os ganhos da rede neural foram extremamente conservadores (indo de 74,99% para 76,26%). A fragmentação de ponteiros na *heap* gera acessos não lineares que dificultam a convergência e extração de correlações sólidas pela função de ponderação do *Perceptron*, fazendo com que o algoritmo opere próximo ao limite do LRU básico.

5.4 Escalabilidade de Hardware e o Custo da Associatividade

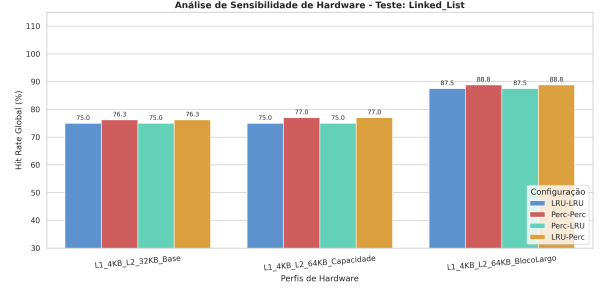
Um dos aspectos críticos na transição de algoritmos históricos para preditores neurais em hardware é a escalabilidade do sistema em relação ao grau de associatividade da cache (*NUM_WAYS*). Em um esquema LRU tradicional, a lógica de controle cresce de forma relativamente contida com a adição de novas vias, dependendo primariamente de contadores ou bits de estado.

Em contrapartida, o algoritmo AIRA exige uma avaliação estritamente paralela para não travar o fluxo de dados. Para uma cache associativa por conjunto de N vias, o hardware precisa instanciar simultaneamente N blocos de inferência do *Perceptron*, além de uma árvore de comparadores de N entradas para avaliar todos os *scores* gerados em um único ciclo e eleger o bloco vítima.

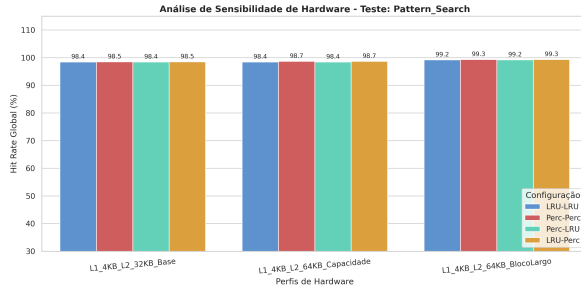
Consequentemente, escalar uma cache L2 de 4 vias para 8 ou 16 vias impõe um crescimento quase linear no consumo de Elementos Lógicos (LEs) e uma expansão logarítmica no atraso combinacional (*combinational delay*) da árvore de comparadores. Este fenômeno acentua a degradação da frequência máxima de operação estimada (F_{max}). Tais restrições confirmam que a adoção da Inteligência Artificial embarcada oferece retornos decrescentes de desempenho se a associatividade for irrestrita. O projeto evidencia que o *sweet spot*



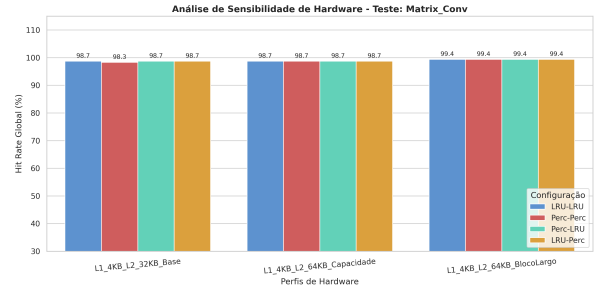
(a) Comportamento em *Streaming*.



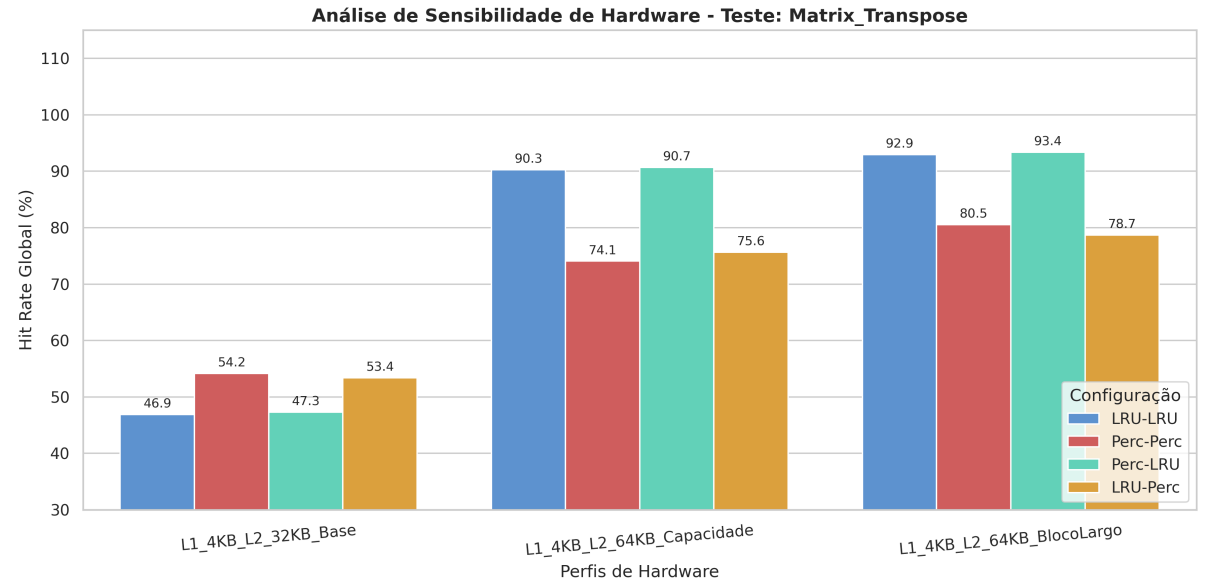
(b) Comportamento em *Linked List*.



(c) Comportamento em *Pattern Search*.



(d) Comportamento em *Matrix Convolution*.



(e) Comportamento em *Matrix Transpose*.

Figura 7: Análise de sensibilidade comparando as taxas de acerto sob diferentes cargas e vias.

Tabela 4: Taxas de Acerto (Hit Rate) por Benchmark na Configuração L1 4KB / L2 32KB

Benchmark	Configuração (L1 - L2)	Hit Rate L1 (%)	Hit Rate L2 (%)	Hit Rate Global (%)
<i>Linked List</i>	LRU - LRU	49,99	49,99	74,99
	LRU - Perceptron	49,99	52,53	76,26
	Perceptron - LRU	49,99	49,99	74,99
	Perceptron - Perceptron	49,99	52,53	76,26
<i>Matrix Convolution</i>	LRU - LRU	97,44	49,98	98,72
	LRU - Perceptron	97,44	49,98	98,72
	Perceptron - LRU	88,66	88,70	98,72
	Perceptron - Perceptron	88,66	84,90	98,29
<i>Matrix Transpose</i>	LRU - LRU	43,75	5,56	46,87
	LRU - Perceptron	43,75	17,10	53,37
	Perceptron - LRU	43,81	6,16	47,27
	Perceptron - Perceptron	43,81	18,47	54,19
<i>Pattern Search</i>	LRU - LRU	96,87	49,99	98,44
	LRU - Perceptron	96,87	50,92	98,47
	Perceptron - LRU	96,87	49,99	98,44
	Perceptron - Perceptron	96,87	50,92	98,47
<i>Streaming</i>	LRU - LRU	87,69	49,97	93,84
	LRU - Perceptron	87,69	61,18	95,22
	Perceptron - LRU	88,02	49,89	94,00
	Perceptron - Perceptron	88,02	65,96	95,92

da arquitetura reside em manter um grau de associatividade moderado, onde o ganho na taxa de acerto compense o alto custo em área inerente ao modelo neural paralelo.

6 Análise Crítica e Conclusões

A síntese lógica isolada do módulo de cache, direcionada à arquitetura de um FPGA Cyclone III, comprovou a viabilidade prática do hardware preditivo baseado em *Perceptron* para substituição de blocos. A transição de um algoritmo puramente histórico (LRU) para uma política que correlaciona contexto de instrução demonstrou ganhos consistentes em estresse de memória, particularmente em *Matrix Transpose* (+7,32% de ganho máximo).

Contudo, os relatórios de síntese evidenciam um claro *trade-off* estrutural. A degradação da F_{max} estimada de 90 MHz para aproximadamente 30 MHz observada se deve às longas cadeias combinacionais na árvore de somadores do *Perceptron*. Isso sugere a imperativa necessidade de técnicas de *pipelining* no *datapath* preditivo em trabalhos futuros antes de sua integração em um *pipeline* de processamento complexo. Em suma, o AIRA provou ser superior logicamente, mas exige arquiteturas de suporte físico adequadas para não gargalar o tempo de ciclo do núcleo.

Referências

- [1] D. A. Jiménez and C. Lin. *Dynamic Branch Prediction with Perceptrons*. Department of Computer Sciences, The University of Texas at Austin, 2001.
- [2] E. Teran, Z. Wang, and D. A. Jiménez. *Perceptron Learning for Reuse Prediction*. Texas A&M University / Intel Labs, IEEE, 2016.